

Smart Link Manager - Day 1 (Deep Learning Notes)

Why We Chose This Project

Goal: Build a Smart Link Manager. Final vision: 1. User pastes a URL. 2. App generates a short URL. 3. AI generates summary. 4. AI generates tags. 5. Data is stored in database. 6. User can view saved links. Today's work focused on building the frontend foundation. Without this foundation, AI, databases, and backend logic cannot be connected.

Creating the Project Folder

Command: `mkdir smart-link-manager` Why? A software project needs a dedicated workspace. The folder acts as the root directory. If we did not create a dedicated folder: - Files would become scattered. - GitHub would become messy. - Frontend and backend could mix together. Think of the folder as the plot of land before building a house.

Opening VS Code

Command: `code .` The dot means 'current directory'. Purpose: - Open the project folder. - Manage files. - Run terminal commands. - Write code. Without VS Code, we could still write code in Notepad, but development becomes very difficult.

What is Node.js?

We checked: `node -v` Node.js allows JavaScript to run outside the browser. Normally: JavaScript -> Browser With Node: JavaScript -> Browser + Computer Why do we need Node? Because React tools, package managers, backend servers and build systems run on Node.

What is npm?

We checked: `npm -v` npm = Node Package Manager. Purpose: - Download libraries. - Manage project dependencies. - Install React and other tools. Analogy: Java -> Maven/Gradle Python -> pip Node -> npm

What is Git?

We checked: `git --version` Git is a version control system. Purpose: - Track changes. - Collaborate. - Push projects to GitHub. Later our entire project history will be managed through Git.

Creating React Using Vite

Command: `npm create vite@latest frontend` Question: Why not create every file manually? Answer: React projects require: - Folder structure - Development server - Configuration files - Build tools Vite automatically creates all of this. Think of Vite as a project generator.

Frontend vs Backend

Frontend: What users see. Examples: - Buttons - Inputs - Text - Cards Backend: Hidden logic. Examples: - Database operations - URL generation - Gemini API calls For our project: Frontend: User enters URL. Backend: Generates short links and AI summaries.

npm install

Command: `npm install` What happens? `package.json` contains dependency information. Example: React Vite `npm install` reads `package.json` and downloads everything. Without `npm install`: The

React application cannot run.

What is React?

React is a JavaScript library used to build user interfaces. React's philosophy: State changes ↓ UI updates automatically Instead of manually updating the screen, we update data and React updates the UI.

What is a Component?

Code: `function App() {}` A component is a reusable UI block. Mental model: Java: `class Student {}` React: `function App() {}` The App component is responsible for generating the screen shown to the user.

What is JSX?

Example: Smart Link Manager Looks like HTML. Actually: HTML-like syntax written inside JavaScript. React converts JSX into JavaScript behind the scenes. Why use JSX? It makes UI easier to read and write.

Input Box and Button

We added: Purpose: The user needs a place to enter URLs. Without input: No URL can be collected. Without button: The generation process cannot start.

React State

Code: `const [url, setUrl] = useState("");` Definition: State = Data remembered by React. Why was state needed? Because the application must remember what URL the user typed. Without state: The application would forget the entered URL.

Understanding useState Deeply

Code: `const [url, setUrl] = useState("");` Initially: `url = ""` After typing: `url = "https://leetcode.com"` Breakdown: `url`: Current value. `setUrl`: Function that changes the value. `useState`: Creates memory for React. Important: Calling `setUrl` not only changes data but also triggers a re-render.

What is onChange?

Code: `onChange={(e) => setUrl(e.target.value)}` Definition: An event handler. When does it run? Whenever the input content changes. Examples: - Typing - Deleting - Pasting text Why needed? React must know when the user changes the input.

What is the Event Object (e)?

Code: `(e)` React automatically creates an event object. This object contains information about the event. Examples: - Which element triggered it? - What key was pressed? - Current value? React passes this object into our function automatically.

What is e.target?

`target` = HTML element that triggered the event. In our project: triggered the event. Therefore: `e.target` refers to the input element itself.

What is e.target.value?

Suppose user types: `https://leetcode.com` Then: `e.target.value` contains: `https://leetcode.com` This is how React reads the text inside the textbox.

Why Did We Use `setUrl(e.target.value)`?

Purpose: Read textbox value ↓ Store it in React state Without this line: React would never know what the user typed. Our entire Smart Link Manager depends on collecting URLs.

What is Re-rendering?

Most important concept today. When state changes: `setUrl(...)` `setSummary(...)` `setError(...)` React runs the component again and updates the screen. Mental model: State changes ↓ React re-renders ↓ UI updates This is React's core idea.

Conditional Rendering

Code: `{summary && {summary}}` Meaning: If summary exists: Show paragraph. Else: Show nothing. Why useful? We don't want empty summary boxes when no summary exists.

Validation

Problem: User clicks Generate without entering URL. Solution: `if(url.trim() === '')` Show error message. Why needed? Real applications must validate user input before processing requests.

What is `trim()`?

Example: `' '` contains spaces. `trim()` removes spaces from beginning and end. Result: `''` This prevents users from bypassing validation using spaces.

Error State

State: `const [error, setError] = useState('');` Purpose: Store error messages. Example: Please enter a URL Without a separate error state, users would not know why generation failed.

Loading State

State: `const [loading, setLoading] = useState(false);` Purpose: Show progress while work is happening. Future: Generate ↓ Backend ↓ Gemini ↓ Summary Users need feedback during waiting.

`setTimeout`

Code: `setTimeout(() => {}, 2000)` Meaning: Wait 2000 milliseconds (2 seconds) then execute code. We used it to simulate waiting for an AI response.

Biggest Lesson of Day 1

Everything in React revolves around: STATE User Input ↓ State ↓ Re-render ↓ UI Update If you truly understand this flow, React becomes much easier.